

THE BUILD, STEP BY STEP

# How We Built Counselor NIDS

Every step of the project in plain words, from reproducing a research paper to a live intrusion detection system, with short technical notes for readers who want the details.

---

Final-year thesis project · October 2026

## Before we start: a few words explained

| Word                             | What it means here                                                                                                                                           |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Network traffic, packets         | Everything computers send each other is cut into small pieces called packets.                                                                                |
| Connection (flow)                | All the packets of one conversation between two computers, for example your browser loading one web page. The system judges connections, not single packets. |
| Intrusion detection system (IDS) | A program that watches traffic and raises an alarm when something looks like an attack. It doesn't block anything; it's the burglar alarm, not the lock.     |
| Machine learning (the AI)        | Programs that learn what attacks look like from many examples, instead of being told rules by a person.                                                      |
| Snort                            | A well-known IDS that uses hand-written rules ("signatures") describing known attacks, a bit like a virus scanner.                                           |
| Dataset                          | A big recording of network traffic in which researchers marked which parts are attacks. We used three public ones: NSL-KDD, CICIDS2017 and CSE-CIC-IDS2018.  |
| False alarm                      | The system says "attack" about normal traffic. Too many false alarms and people stop trusting the system.                                                    |
| Detection rate                   | Out of all real attacks, the share the system caught.                                                                                                        |

## The idea in one paragraph

A 2019 research paper (Quincozes et al.) proposed an intrusion detection system made of several AI "detectors", each an expert in some attacks. When one is unsure, it asks the others for advice, like a doctor asking colleagues. The paper calls the advisors **counselors**. We first rebuilt that idea, then turned it into a real system that watches live network traffic next to Snort. Finally, for the thesis, we let Snort act as a counselor and teacher, so the AI can adapt to a new network on its own.

## Part 1 — Rebuilding the research paper

---

### Step 1 Setting up the project and the data

We created the project and downloaded the paper's datasets: NSL-KDD (an older, classic dataset) and CICIDS2017 (modern traffic recorded in a lab, with attacks such as floods, password guessing and port scans). We wrote "loaders": small programs that read these files and clean them up, for example removing broken values.

#### TECHNICAL NOTE

Python 3 with pandas and scikit-learn. CICIDS2017's "MachineLearningCSV" files have 78 measurements per connection; some contain `NaN` or `Infinity`, and the web-attack labels contain an odd non-UTF-8 dash character that had to be handled.

### Step 2 Teaching one detector (the paper's Algorithm 1)

A detector doesn't rely on one AI method. It trains five different ones, then sorts the training traffic into groups of similar connections. For each group it remembers which method was best. Different methods are good at different kinds of traffic, so each group gets its own expert.

#### TECHNICAL NOTE

Classifiers: naive Bayes, three decision trees (standing in for Weka's J48, REPTree and RandomTree) and a random forest. Grouping uses K-Means clustering. Per cluster, the most accurate classifier and any within 0.1% of it are selected. We also log-compressed the numbers before scaling: without it, single extreme connections got whole clusters to themselves.

### Step 3 Judging new traffic (Algorithm 2) and asking counselors

For a new connection, the detector finds its group and lets that group's chosen methods vote. If they agree, done. If they disagree, the detector asks the other detectors (its counselors) what they think, and takes the advice of one that has been at least 90% accurate.

#### TECHNICAL NOTE

Counselors answer from their own recent confident decisions about the same connection. The paper matches by time (a 2-second window); we match the exact connection, which is stricter. Advice below the accuracy threshold is ignored.

#### Step 4 Testing it like the paper, and finding a weakness

We reproduced the paper's experiments. The paper's method reached about 83% accuracy per detector on CICIDS2017; the paper itself reported 88.36% and 74.57%. We then found a blind spot. A detector that never learned an attack, say port scans, is not "unsure" about it: it is confidently wrong, so it never asks for advice. We added a **cross-check**: even a confident "normal" is checked with the counselors. Accuracy jumped to **99.88%**.

##### TECHNICAL NOTE

Scenario 2 (CICIDS2017), mean of 3 runs: paper's method 83.01% / 83.77%, with cross-check 99.88% / 99.88%. Settings were tuned on a separate validation set so the test stayed fair. We also tested self-learning (detectors retraining on the advice they received): with cross-check, a detector's own accuracy on an attack it never learned rose from 32.5% to 99.9%.

## Part 2 — From experiment to a running system

---

### Step 5 Splitting the system into cooperating programs

In the paper's design, each part is a separate program: one captures traffic (the **Extractor**), one hands it out (the **Observer**), and each detector runs on its own. They talk through a fast message board called Redis. If one part crashes, the others keep going, and parts could even run on different computers.

#### TECHNICAL NOTE

Redis streams carry batches of connections, each detector has an inbox, and advice requests and replies are Redis lists. The `nids` command starts each part: `nids extract`, `nids observe`, `nids detect`.

### Step 6 A web dashboard

We built a website that shows what the system sees as it happens: how many connections were checked, which were flagged, and why. It has a login page and updates live.

#### TECHNICAL NOTE

Backend: Django with Django REST Framework and Channels (WebSocket live feed every second). Frontend: Next.js (React, TypeScript) with charts. Login uses httpOnly cookies. JavaScript can't read, a CSRF guard header, and a limit of 10 login attempts per minute.

### Step 7 Adding Snort next to the AI

We made Snort watch the same traffic. When Snort raises an alarm, the system finds the exact connection it is about and asks the AI what it thinks. Each alarm becomes "Confirmed" (both agree), "Disputed" (only Snort), or "AI only". This is useful because the two make different mistakes.

#### TECHNICAL NOTE

Snort writes alerts as JSON. A "flow index" in Redis stores every connection by its two addresses, ports and protocol, plus its start and end time. An alert is matched to the connection that was open at that moment. Port-scan alerts are matched to all connections between the two hosts within  $\pm 30$  s.

### Step 8 One command to run everything

Instead of starting ten programs by hand, `./start.sh setup` installs everything once, and `./start.sh live wlan0` starts the whole system on your Wi-Fi card, plus the dashboard.

## Step 9 The big surprise: the AI caught nothing on live traffic

When we pointed the system at real traffic, the AI caught **0%** of password-guessing attacks. The reason: the public datasets were measured with one program (the Java CICFlowMeter), while live capture uses a different one (the Python cicflowmeter). They measure connections differently, for example whether packet sizes include headers. To the AI, the same attack looked completely different.

The fix: train new **live detectors** on the datasets' original recordings, measured with the same program live capture uses. We downloaded only the parts we needed from CSE-CIC-IDS2018, about 1.3 GB out of hundreds.

### TECHNICAL NOTE

71 of 154 feature medians differed between the two flow meters. We fetched single hosts' captures from the dataset's ZIP archives with HTTP range requests, found each attack's time window in the packets themselves (the CSV timestamps are a 12-hour clock without AM/PM), and labelled connections by their attacker-victim pair.

## Step 10 Cutting false alarms on live traffic

Live, the AI raised many false alarms. We found two causes. First, connections that carry no data, such as port-scan probes and failed connections: the AI never learned them and just guessed (one scan alone caused about 50,000 alarms). They now go to Snort only. Second, "best guesses" when the AI's methods disagreed and nobody could advise: they caused most false alarms but almost no real detections, so they now count as normal.

## Step 11 Making the dashboard understandable

We rewrote every label in plain language ("AI" and "rule checker" instead of jargon), and added:

- a severity for each alert (Critical to Low) with an explanation of what it means;
- an "attackers" view that groups alarms by who sent them (for example scan, then password guessing);
- desktop notifications for serious alarms, and a printable report;
- a ready-made firewall command to block an attacker, which never runs anything by itself;
- offline checks against public lists of known-bad addresses.

## Part 3 — Making it reliable for real use

---

### Step 12 Live first, and surviving Wi-Fi drops

We made live monitoring the dashboard's main view and moved test recordings to their own page. Capture now waits when the network drops (for example Wi-Fi disconnects) and resumes on its own. Connections are also sent to the AI every 2 seconds instead of waiting for a batch of 500, so a quiet home network gets answers quickly.

### Step 13 A third detector: botnets

A botnet is a group of infected computers that secretly check in with the attacker's control server. We trained a **Botnet detector** on infected machines from CSE-CIC-IDS2018. On an infected machine it had never seen, it caught 99.8% of the botnet traffic.

#### LESSON LEARNED

Retraining all three detectors at once made website-attack detection fall from 88% to 23%, because there are very few website-attack examples. So we trained only the new detector and kept the proven ones.

### Step 14 Safe retraining: the promotion gate

We found that retraining could randomly give good or bad detectors: false alarms anywhere from 0 to 921 on the same data. One cause was a sorting detail (many connections start in the same second, and their order changed). We fixed that. We also added a **promotion gate**: new detectors are tested against the installed ones on the same test traffic, and only replace them if they are not worse.

#### TECHNICAL NOTE

Stable sort in the time split. The gate fails if false alarms rise or any attack type drops by more than 2 points. The installed models ship in the repository with SHA-256 checksums; model files are pickles, which run code when loaded, so live mode refuses any file that doesn't match.

### Step 15 Measuring false alarms on your own network

Lab numbers don't tell you how noisy the system is at home. We added an **alert journal** that keeps every alarm on disk for a year, and a dashboard panel that shows false alarms per 1,000 connections. Before attacking your own machine for a test, you click "I'm starting an attack test", so those alarms aren't counted as false.

## Step 16 Snort was blind to website attacks, and why

We tested Snort on a recording full of website attacks: it raised **zero** alarms. About 40% of the packets had "wrong" checksums, a small error-check number. Modern network cards fill that number in at the last moment, so recordings and your own computer's outgoing traffic show it as wrong, and Snort was silently throwing those packets away. After turning that check off and writing new rules (SQL injection, cross-site scripting, website password guessing), Snort caught **90.8%** of the attacker's connections, with no false alarms on normal traffic. We also silenced community rules that only report ordinary things such as pings.

## Step 17 A clock bug that hurt Slowloris detection

Slowloris is an attack that keeps many connections open by sending data very slowly. When the flow meter read a *recording*, it ended connections using the computer's clock, not the recording's. A recording from 2018 looks years old, so connections were cut at random points. The AI had learned from chopped-up Slowloris connections. After fixing it and retraining the Flood detector, Slowloris detection rose from 56% to 85% on the test data, and overall detection from 93.5% to 99.1%.

### TECHNICAL NOTE

cicflowmeter 0.5.0 runs a background thread that expires flows using `time.time()`. Reading a recording, we stop that thread, so flows end by packet time only (on FIN, after 240 s idle, or past 90–120 s), as in live capture. Output is now identical on every run.

## Step 18 Running it like a real product

For a server that should run all the time:

- **Services:** each part runs as a system service that starts at boot and restarts by itself if it crashes.
- **Least privilege:** nothing runs as the all-powerful root user. Capture gets only the permission to read network traffic.
- **Health checks:** every part sends an "I'm alive" signal. The dashboard shows a green or red chip for each, and a check runs every minute.
- **Logs** are rotated so they don't fill the disk.

### TECHNICAL NOTE

systemd units with `Restart=on-failure`, `AmbientCapabilities=CAP_NET_RAW CAP_NET_ADMIN`, `NoNewPrivileges`; heartbeats in a Redis hash; `nids health` exits with 1 if a part is silent for 15 s.

## Step 19 Cleaning up: product only

The paper experiments, notebooks, test suites and old dataset-mode detectors had served their purpose, so we removed them from the project, along with 7 GB of old data. What remains is what the live system and its retraining need. We checked that the whole system still runs end to end afterwards, and drew the system-design and logic-flow diagrams.

## Part 4 — The thesis contribution

---

### Step 20 The problem we chose

Studies show that an AI detector trained on one network often falls to about chance accuracy on another, and every real deployment is "another network". Labelling the new network's traffic by hand is too expensive. Our idea: Snort's rules work the same everywhere, so let Snort guide the AI on the new network.

### Step 21 Snort becomes a counselor

Until now, detectors only asked each other. Now Snort is also a counselor: when a detector is unsure, or double-checks a "normal" verdict, Snort's opinion on that connection counts as advice. Snort only speaks when it raised an alarm; staying silent doesn't mean "safe", because Snort can't see some attacks, such as floods.

### Step 22 Learning from agreement

When Snort and the AI *independently* say "attack", or Snort is silent and every detector is sure a connection is normal, we treat that as a label. The detectors retrain on these labels. A fifth of them is held back as a quiz, and a retrained detector is only kept if it scores at least as well on the quiz. No human labels are used.

### Step 23 Adaptive trust

How much each counselor is trusted used to be fixed in the lab. Now it is re-measured on the new network:

- each detector's accuracy for each group of similar connections;
- each Snort rule's reliability, from how often the AI agrees with it.

A rule that keeps firing on traffic the AI is sure is normal loses its say.

### Step 24 The experiment

We train the detectors on one lab (2018), then let them watch the other lab's traffic (2017) in time order, as if newly deployed, and check after each chunk how well they do on a fixed test set. Each feature is switched on one at a time, and Snort alone and AI-or-Snort serve as baselines, so the thesis can show exactly what each part adds. We also measure how often the agreement labels were actually right.

#### STATUS

Everything is built. The results need the 2017 recordings (about 50 GB), which are being downloaded.

## What we learned

---

- **Lab scores can lie.** A model can be 99% accurate on its dataset and 0% on live traffic, just because the traffic was measured differently.
- **Small details break big systems:** a checksum setting blinded Snort, a clock setting distorted training data, a sorting detail made results random.
- **Two different methods are better than one.** The AI catches floods and botnets that rules miss; rules catch website attacks the AI barely knows.
- **Safety nets matter:** a promotion gate, checksum-verified models and health checks stop silent failures.
- **Measure on the real network.** The false-alarm journal answers the question lab tests can't.

## Tools we used

---

| Area        | Tools                                                                                                                                   |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| AI and data | Python, pandas, NumPy, scikit-learn (K-Means, naive Bayes, decision trees, random forest), joblib                                       |
| Traffic     | Python cicflowmeter (with our fixes), Snort 3, tcpdump, our own pcap reader                                                             |
| Messaging   | Redis (streams, lists, hashes, sorted sets)                                                                                             |
| Web         | Django, Django REST Framework, Channels, Daphne; Next.js, React, TypeScript, Recharts                                                   |
| Operations  | systemd, Docker (dashboard only), bash ( <code>start.sh</code> )                                                                        |
| Data        | NSL-KDD and CICIDS2017 (paper reproduction), CSE-CIC-IDS2018 raw captures (live detectors), CICIDS2017 raw captures (thesis experiment) |